Instructor Handout 4: System Programs and OS Structures

Lecture 4 — Monolithic, Layered, Microkernel, Modular, Virtual Machine

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

4.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 2: §2.5 (system programs, p. 66), §2.6 (design and implementation: goals, mechanism vs. policy, implementation language), §2.7 (OS structure: simple, layered, microkernels, modules, pp. 70–76), §2.8 (virtual machines, p. 76); §2.5–2.8 span pp. 66–84. Primary anchor for every topic. *Page-verified* via the course index (section-level pages; the index does not give per-subsection pages inside §2.7, so cite the range).
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 2 §2.3 (system structure among the major achievements, p. 64) and Ch. 4 §4.3 (microkernels, p. 179), for the fuller list of microkernel benefits. *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 1 (pp. 1–34), for Linux as a monolithic kernel with loadable modules. *Page-verified* at chapter level.
- **Prepare one live demo (optional).** On Linux, `lsmod` lists loaded modules and `type cd` / `type ls` in `bash` shows one built-in and one file. Both make the abstract claims concrete in under a minute.
- **Draw the five structure figures from memory** (Figures 4.1–4.5). The lecture is a comparison; you must be able to put two side by side on the board without notes.
- **Assessment context.** The syllabus schedules Class Test 1 (Unit I part) around this lecture; the comparison table (Table 4.2) and the crossing-count model are the most testable items.

Teaching note: The thesis for the hour: *isolation is bought with boundary crossings.* Every structure is a point on that trade-off, and the buggy driver is the probe that reveals where each point lies. Keep a five-row "blast radius" table in a corner of the board and fill one row per structure as you go.

4.2 Opening: One Bad Driver

Pose the question before naming any structure: a network-card driver dereferences a `NULL` pointer on a rare packet. Does the machine crash, does only the network stop, or does nobody notice? The answer is not in the driver's code; it is in how the operating system is organized — whether the driver runs in kernel mode or user mode, inside the kernel or beside it. Last lecture was the system-call doorway; this lecture is what sits on each side of it. The roadmap has five stops: system programs, design principles, kernel structures (simple, monolithic, layered, microkernel, modular), virtual machines, and a comparison.

Board: the empty blast-radius table. *On the board, in order:* (i) draw five rows labelled Monolithic, Layered, Microkernel, Modular, VM; (ii) add two columns, "driver runs in" and "blast radius"; (iii) leave every cell blank. *Say:* "By the end of the hour every cell is filled, and you will be able to fill them in an exam without me."

Board: the poll. *On the board, in order:* (i) write the three options — whole machine / network only / nobody notices; (ii) take a show of hands; (iii) write the counts and leave them up. *Say:* "Most of you said 'whole machine.' That is right for three of the five structures and wrong for two."

> **Teaching note:** Students who have seen a Windows blue screen will answer "whole machine" confidently. Do not correct them yet; the microkernel and VM rows are where the correction lands, and the poll counts make the reversal memorable.

4.3 System Programs

Most of what a user calls "the operating system" is not the kernel. It is a set of ordinary programs that make system calls.

Definition (System program). *A **system program** (system utility) is a program shipped with the operating system that gives users a convenient environment for program development and execution. It runs in user mode and obtains every privileged service through system calls.*

`ls`, `cp`, `cc`, `vi`, the shell, and the login program are all system programs; the user's view of the OS is defined by them, not by the system calls. Silberschatz (Ch. 2 §2.5, p. 66) gives six categories: file management (`cp`, `mv`, `rm`, `ls`); status information (`date`, `who`, `df`); file modification (`vi`, `grep`, `sed`); programming-language support (`cc`, `as`, `gdb`); program loading and execution (`ld`, the loader); and communications (`ssh`, `mail`, `ftp`). Application programs — browsers, word processors, databases — often ship with the system but are not part of the OS proper.

Example 4.3.1 (The deck's six commands)**.** *`ls -l`: file management. `date`: status information (it reads the clock through an information-maintenance call). `vi notes.txt`: file modification. `cc prog.c`: programming-language support. `ld prog.o`: program loading and execution. `ssh server`: communications.*

Board: classify by voting. *On the board, in order:* (i) write the six category names across the top; (ii) write `ls -l`, `date`, `vi`, `cc`, `ld`, `ssh` down the side; (iii) for each, ask for a category before you place a tick. *Say:* "Every command you have typed this semester lives in one of these six columns."

Board: a second set for practice (the Notes' exercise set). *On the board, in order:* (i) keep the columns; (ii) write `cp`, `who`, `grep`, `gdb`, `mail`; (iii) have pairs classify them in thirty seconds, then reveal: file management, status information, file modification, language support, communications.

Concepts That Click

Teaching note: The common confusion is between the two taxonomies. `date` is a status-information *program* that makes an information-maintenance *call*. Write "program category ≠ call category" on the board when `date` comes up, and point at Lecture 3's six call categories.

The command interpreter

There are two ways to build a shell (Silberschatz, Ch. 2 §2.2). With built-in commands, the interpreter contains each command's code and grows with every command. With commands as system programs (UNIX), `rm file.txt` makes the shell find a file named `rm`, load it, and pass `file.txt`; new commands are new files and the shell stays small. UNIX uses the second design for most commands but keeps a few built-ins such as `cd`.

Board: why `cd` is built in. *On the board, in order:* (i) draw a box "shell, cwd = /home" ; (ii) draw an arrow labelled `fork` to a child box "cwd = /home"; (iii) in the child, write `chdir("/tmp")` and change its cwd to /tmp; (iv) cross the child out (it exits) and point at the parent, still /home. *Say:* "The directory changed — in a process that no longer exists."

Board: the same argument for `exit` (a second instance). *On the board, in order:* reuse the two boxes; in the child write `exit()`; cross out the child; the shell's `wait` returns and the prompt reappears. *Ask:* "So can `exit` be a separate program?" (No — it would end only the child.)

4.4 Design Principles

Users want an OS that is convenient, easy to learn, reliable, safe, and fast; builders want one that is easy to design, implement, and maintain, flexible, reliable, error-free, and efficient. The goals conflict, so there is no single best OS, only a best fit to requirements.

Definition (Mechanism vs. policy). *A* ***mechanism*** *determines* how *to do something; a* ***policy*** *determines* what *will be done. Separating them lets a policy change without rewriting the mechanism.*

Table 4.1 is the deck's worked example. The timer mechanism of Lecture 2 is unchanged whether the quantum is 10 ms or 100 ms; only a parameter changes. A microkernel pushes the idea to its limit: the kernel keeps mechanisms, servers hold policy.

Situation	Mechanism (how)	Policy (what)
CPU protection	the timer interrupts the CPU after a set interval	how long the interval is for each user
Scheduling	a context switch from one process to another	which process runs next
File protection	the kernel checks a permission bit before `open`	who is granted which bits
Our driver	loading code into the kernel at run time	which drivers are trusted enough to load

Table 4.1: Mechanism versus policy (deck, "Separate the Mechanism From the Policy").

Board: the "change one thing" test. *On the board, in order:* (i) write "quantum = 10 ms"; (ii) cross it out and write "100 ms"; (iii) ask what code changed — none, only a parameter; (iv) label the timer *mechanism* and the number *policy*. *Say:* "If changing the decision forces you to rewrite code, mechanism and policy were tangled."

Concepts That Click

Board: file protection as a second instance. *On the board, in order:* (i) draw a file with permission bits `rw-r-----`; (ii) write "kernel checks bits on `open`" (mechanism); (iii) change the bits with `chmod` (policy changes, kernel code does not).

Implementation language: early systems were written in assembly; Linux and Windows are mostly C with small assembly parts (trap entry, context switch, device registers). A high-level language is faster to write, easier to debug, and easier to port; the possible slowdown matters less than algorithms and data structures, which are tuned after profiling.

> **Teaching note:** Students often think "policy" means a written rule or a legal document. Anchor it as "the parameter or decision," and mechanism as "the code that obeys it."

4.5 Simple and Monolithic Structure

MS-DOS (Figure 4.1) was written for small machines; its levels are not well separated, and applications can call BIOS routines and write to the display directly because the Intel 8088 had no dual mode and no hardware protection. One bad program can crash the whole system.

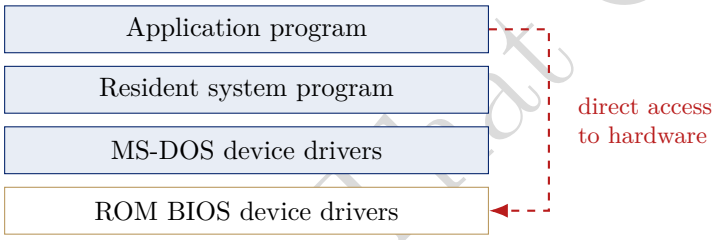


Figure 4.1: MS-DOS layer structure (deck figure, verbatim) — Silberschatz et al., Ch. 2 §2.7, pp. 70–76.

How to draw it: the coordinate map gives the build order — four equal boxes from the top (application, resident system program, MS-DOS drivers, ROM BIOS drivers), then the dashed bypass from the application's right edge down to the BIOS's right edge, then the label "direct access to hardware" to the right of the dashed line. Draw the bypass last and in a different colour: it is the whole point.

Traditional UNIX (Figure 4.2) has two parts, kernel and system programs. Everything between the system-call interface and the hardware interface is one program in kernel mode and one address space: fast, because subsystems call each other as functions, and hard to maintain.

How to draw it: top to bottom — users, system programs, a dashed system-call interface, one tall kernel box (write the subsystems inside, with "device drivers" last and underlined), a dashed hardware interface, hardware. Say "one address space" as you close the kernel box.

Board: the monolithic driver trace (deck Running Example 1). *On the board, in order:* (i) circle "device drivers" inside the kernel box; (ii) write the four steps: driver in kernel mode → `NULL` dereference → fault in kernel mode, nothing more privileged to contain it → kernel cannot tell what was corrupted, so it panics; (iii) fill the table row: kernel mode / whole machine.

Board: contrast with MS-DOS. *On the board, in order:* (i) beside the UNIX figure, redraw the four MS-DOS boxes small; (ii) ask: "In MS-DOS, can an *application* bug crash the machine?" (yes — no dual mode); "In UNIX?" (an application bug, no; a driver bug, yes). *Say:* "UNIX added a wall between applications and kernel, but not between the parts of the kernel."

Concepts That Click

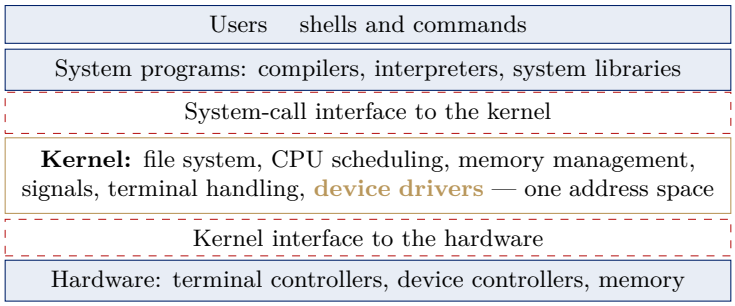


Figure 4.2: Traditional UNIX structure (deck figure, verbatim) — Silberschatz et al., Ch. 2 §2.7, pp. 70–76.

> **Teaching note:** Students conflate "simple" and "monolithic." The distinction to stress: MS-DOS lacks *hardware* protection between applications and the OS; UNIX has it, but its kernel is one undivided privileged program.

4.6 The Layered Approach

The OS is a stack of layers from 0 (hardware) to N (user interface); layer i uses only layers below i and hides its data structures from those above (Figure 4.3). The payoff is bottom-up debugging: once layer $i-1$ is verified, a bug found while testing layer i must be in layer i.

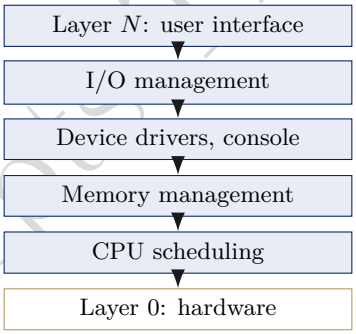


Figure 4.3: A layered OS (deck figure, verbatim) — Silberschatz et al., Ch. 2 §2.7, pp. 70–76; Stallings, Ch. 2 §2.3, p. 64.

How to draw it: stack the six boxes from the bottom up (hardware first, labelled "layer 0"), then add the short downward arrows in the gaps one at a time, saying "calls only the layer below" each time.

Example 4.6.1 (Where does the backing-store driver go? (deck))**.** *Rule: if A calls B, B is lower. (1) Memory management swaps pages, so it calls the driver: driver below memory management. (2) The driver waits for I/O and the CPU should run someone else, so it calls the scheduler: scheduler below driver. (3) On a large system the scheduler's tables may be swapped, so the scheduler calls the driver: driver below scheduler. Steps 2 and 3 contradict: some dependencies have no clean order.*

Board: the ordering puzzle. *On the board, in order:* (i) write three cards: MM, D, S; (ii) draw the arrow MM→D, place D under MM; (iii) draw D→S, place S under D; (iv) draw S→D and ask the class where D goes now. Let the contradiction sit for ten seconds before naming it.

Concepts That Click

Board: a second, solvable instance (assignment-style). *On the board, in order:* (i) four components F, M, D, S with calls F→M, F→D, M→D, D→S; (ii) have students derive the order; (iii) reveal the unique chain $S < D < M < F$. *Say:* "When the calls form a chain, layering is easy; when they form a cycle, it is impossible."

The costs: defining layers is hard, and each layer adds overhead — a request from the top passes through every layer below, each adding a call and possibly copying parameters (an I/O system call traverses I/O → memory → scheduling → hardware). Modern systems use fewer, more functional layers. Running Example 2: layering does not change privilege, so the driver layer is still kernel mode and the `NULL` dereference still crashes the kernel; the win is finding the bug sooner, not surviving it.

> **Teaching note:** The most persistent misconception of the lecture: "upper layers run in user mode." Ask directly, "Which layers in this figure run in kernel mode?" and insist on "all of them." Then fill the table row: kernel mode / whole machine.

4.7 Microkernels

If kernel mode is where one bug becomes everyone's bug, put as little as possible in kernel mode. A microkernel keeps message passing, basic memory management, and CPU scheduling; file systems, drivers, and networking move into user-level servers that exchange messages through the kernel (Figure 4.4). Mach (CMU, mid-1980s) and QNX are the examples.

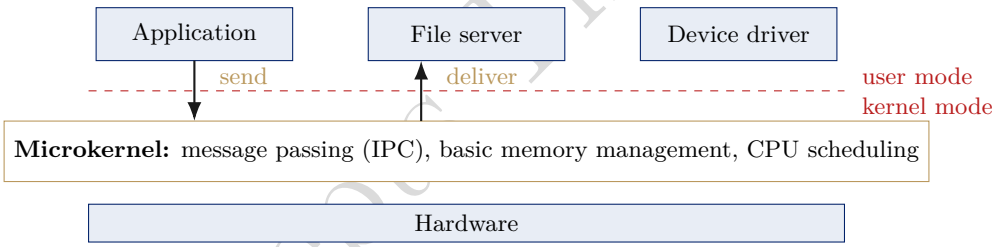


Figure 4.4: Microkernel structure (deck figure, verbatim) — Silberschatz et al., Ch. 2 §2.7; Stallings, Ch. 4 §4.3, p. 179.

How to draw it: three user boxes along the top, then the dashed red mode line, then the wide microkernel box and hardware. Draw the "send" arrow (application down into the kernel) and only then the "deliver" arrow (kernel up into the file server). Ask before drawing the second arrow: "Can the application call the file server directly?"

Example 4.7.1 (Counting boundary crossings (deck))**.** *Each trap is one entry plus one return, 2 crossings. Monolithic read: 1 trap, 2 crossings. Microkernel with a file server only: send + reply, 2 traps, 4 crossings. File server plus driver server: app→fs, fs→drv, drv→fs, fs→app, 4 traps, 8 crossings. In general, a chain of k servers costs $2k$ traps, $4k$ crossings, versus 2 monolithic; with $k = 2$, four times as many. Message copies between address spaces add the "system-function overhead."*

Board: count with $k = 1$ and $k = 2$. *On the board, in order:* (i) draw the app and one server above a line, the kernel below; (ii) draw each message as a down-and-up zigzag through the kernel and count arrows crossing the line (4); (iii) add a second server and redraw (8). *Ask:* "Predict $k = 3$ before I draw it." (12.)

Concepts That Click

Board: what it costs per second (a second instance, from the assignment family). *On the board, in order:* (i) write "0.5 μs per crossing, 100,000 reads/s"; (ii) monolithic: $2 \times 0.5 = 1$ μs per read; (iii) microkernel $k = 2$: $8 \times 0.5 = 4$ μs per read; (iv) difference 3 μs $\times$ 100,000 = 300 ms of CPU per second. *Say:* "Isolation is not free; here it costs almost a third of a CPU."

Benefits: extensible (new service = new server), portable (little hardware-specific code in the kernel), reliable and secure (services are user processes); Stallings (Ch. 4 §4.3, p. 179) adds uniform interfaces, flexibility, and distributed-system support. Cost: performance. Running Example 3: the driver is a user process; the `NULL` dereference is an ordinary page fault, the kernel kills only that process and can restart it. Table row: user mode / the driver only. The Windows NT history (Socratic Check 2) closes the topic: NT began as a layered microkernel and moved services back into the kernel to cut messages and mode switches, giving up isolation.

> **Teaching note:** Students hear "micro" and conclude "fast." Ask them to count crossings before you state the cost; the counting example is the antidote.

4.8 Modules and Hybrid Systems

A modular kernel is a small core plus loadable modules linked in at boot or run time without recompiling. Modules have defined interfaces like layers, may call one another like microkernel servers, but need no messages, because all run in kernel mode. Solaris has a core kernel with seven module types: scheduling classes, file systems, loadable system calls, executable formats, STREAMS modules, miscellaneous, and device and bus drivers. Linux is monolithic with loadable modules (Bovet & Cesati, Ch. 1). Hybrids: Linux (monolithic + modules), Mac OS X (Mach microkernel + BSD environment + kernel extensions), Windows (largely monolithic, with microkernel-style user-mode subsystems).

Board: `insmod` (Running Example 4). *On the board, in order:* (i) redraw the UNIX kernel box with a gap; (ii) draw `netdrv.ko` sliding into the gap with the label `insmod`; (iii) ask what mode it now runs in (kernel); (iv) fill the row: kernel mode / whole machine. *Say:* "Modules buy flexibility, not isolation."

Board: the seven Solaris module types as a hub. *On the board, in order:* (i) a circle "core kernel"; (ii) seven spokes, one per module type; (iii) ask which spoke our driver hangs on (device and bus drivers).

> **Teaching note:** "Loadable" sounds like "separate." Run `lsmod` if you can; the modules listed are inside the kernel, sharing its address space.

4.9 Virtual Machines

Push isolation further: run the whole operating system as a guest. The virtual-machine manager gives each guest the illusion of its own processor, memory, and devices; each guest runs a full, unmodified OS (Figure 4.5). Examples: IBM VM/370 (1972), VMware, the Java Virtual Machine.

How to draw it: hardware, then the manager across the full width, then three guest columns, each a kernel box topped by a processes box. Draw all three columns identically, then shade VM 1 for the driver example.

Benefits: isolation (a virus in one VM cannot reach another directly), system development (test a new kernel without disrupting others), consolidation (lightly used servers become VMs on one

Concepts That Click

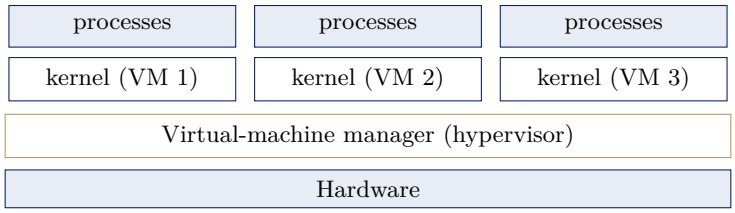


Figure 4.5: Virtual machines (deck figure, verbatim) — Silberschatz et al., Ch. 2 §2.8, p. 76.

machine). Cost: the guest kernel runs de-privileged, so its privileged operations are trapped and emulated by the manager. Running Example 5: the driver is in VM 1's guest kernel; VM 1 panics, the manager and VMs 2 and 3 keep running. Row: guest kernel / one VM.

Board: shade the casualties. *On the board, in order:* (i) the three-column figure; (ii) shade all of VM 1 red; (iii) ask which boxes survive (manager, VMs 2 and 3).

Board: consolidation arithmetic (second instance). *On the board, in order:* (i) write "40 servers, each 5% busy"; (ii) ask how many physical machines at most 50% load need (four, with ten VMs each); (iii) note that a crash in one VM still loses only that VM.

4.10 Comparison and Close

Structure	Driver runs in	Bug's blast radius	Speed	Main strength
Monolithic	kernel mode	whole machine	fastest (calls)	performance
Layered	kernel mode	whole machine	layer overhead	bottom-up debugging
Microkernel	user mode	the driver only	messages, $4k$ crossings	isolation, extensibility
Modular	kernel mode	whole machine	near monolithic	load at run time
Virtual machine	guest kernel	one VM	trap-and-emulate	isolates whole OSes

Table 4.2: One driver bug, five structures (deck comparison slide).

The board table should now match Table 4.2. Return to the opening poll: "whole machine" was right for monolithic, layered, and modular; wrong for microkernel and VM. Socratic Check 3 (flight-control computer with a network-logging driver) has the answer microkernel — the logging server's failure cannot take down control software, at a price of $4k$ crossings instead of 2 — which is why QNX-style microkernels are common in embedded and real-time systems.

Misconceptions to clear explicitly: the shell is part of the kernel (no — a user-mode system program); layered means upper layers are user mode (no — privilege unchanged); modules isolate drivers (no — kernel mode); microkernels are faster because they are small (no — messages add crossings).

> **Teaching note:** End by pointing forward: every structure manages the same unit, the process, and the next lecture introduces it and its process control block (Silberschatz Ch. 3 §3.1; Stallings Ch. 3 §3.1–3.3).

Concepts That Click

4.11 Summary

- **System programs**: six categories (file management, status information, file modification, language support, loading and execution, communications); user mode; built on system calls. `cd` must be a built-in.
- **Mechanism** = how; **policy** = what. Separate them so policies change without code changes.
- **MS-DOS**: no dual mode, applications reach hardware. **UNIX**: kernel + system programs; monolithic kernel, one address space.
- **Layered**: layer i uses only lower layers; bottom-up debugging; ordering can cycle (backing-store driver vs. scheduler); per-layer overhead.
- **Microkernel**: minimal kernel, user-level servers, messages; $2k$ traps $= 4k$ crossings for a chain of k servers vs. 2; isolation and extensibility for speed.
- **Modular**: core + loadable modules, kernel mode, flexibility not isolation. **VM**: whole OSes as isolated guests; trap-and-emulate cost.

Concepts That Click